REPUBLIC OF CAMEROON
PEACE-WORK-FATHERLAND
UNIVERSITY OF BUEA
BUEA, SOUTH-WEST REGION
P.O BOX 63.

RÉPUBLIQUE DU CAMEROUN
PAIX-TRAVAIL-PATRIE
UNIVERSITÉ DE BUEA
BUEA, RÉGION DU SUD-OUEST
B.P. 63.

FACULTY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER ENGINEERING, SE
CEF440: INTERNET PROGRAMMING

TASK 4: SYSTEM MODELLING AND DESIGN

Report by: GROUP 11

Department: Computer Engineering

FACULTY OF ENGINEERING AND TECHNOLOGY
Department of Computer Engineering — Software Engineering

CEF440: Internet Programming

CheckAR — AI-Based Car Fault Diagnosis Mobile Application

System Architecture and Design Report

Prepared by: Group 11

Name	Matricule
FUNWI KELSEA NDOHNWI	FE23A066
NGADANG ASSONFACK P. ESTELLE	FE23A109
ATEH SWIRRI FOFANG	FE23A018
TABOT GHISLAIN OROCK	FE23A146

TABLE OF CONTENTS

1. Introduction

1.1 Background

Modern vehicles are equipped with increasingly sophisticated electronic control systems, sensors, and embedded software that govern everything from engine management to safety systems. While this technological complexity improves vehicle performance and safety, it has simultaneously made self-diagnosis a daunting challenge for the average driver. Most vehicle owners lack the specialized knowledge or equipment required to interpret dashboard warning lights or unusual engine sounds — two of the most common indicators of mechanical or electronic faults.

Traditional diagnostic approaches typically require a physical visit to a workshop, where a trained mechanic uses an On-Board Diagnostics (OBD-II) scanner to read fault codes from the vehicle's Electronic Control Unit (ECU). This process is not only time-consuming and costly, but also inaccessible in situations where a vehicle breaks down in a remote location or outside of normal workshop hours. Furthermore, many vehicle owners in developing regions do not have easy access to well-equipped repair centres, compounding the problem.

In response to these challenges, CheckAR — the Car Fault Diagnosis Mobile Application — was developed as an AI-powered solution that empowers drivers and mechanics to identify and understand vehicle faults using only a smartphone. By capturing a photograph of the vehicle's dashboard or recording a short audio clip of the engine, users can receive instant, intelligent diagnostic reports generated by machine learning models running either on-device or in the cloud. The application further enriches the diagnostic experience by linking identified faults to relevant video tutorials, locating nearby mechanics via map integration, and maintaining a personal maintenance history for each user.

CheckAR bridges the gap between ordinary vehicle owners and professional diagnostic tools by democratising access to automotive intelligence. Its architecture combines Flutter-based mobile development, Firebase cloud services, TensorFlow Lite on-device inference, and a Node.js backend to deliver a robust, scalable, and user-friendly platform suitable for a wide range of vehicles and fault scenarios.

1.2 Aims and Objectives of the Report

The primary aim of this report is to present a comprehensive architectural and functional overview of the CheckAR system. Specifically, this report seeks to achieve the following objectives:

- Present a structured understanding of how CheckAR interacts with external entities, users, and third-party services through clearly defined system boundaries.
- Visualise the flow of information through standardised UML and DFD diagrams that depict system behaviour, process responsibilities, data storage, and actor interactions.
- Provide a detailed technical blueprint — covering context, data flow, use cases, class structure, interaction sequences, and physical deployment — that developers and system architects can reference during implementation and scaling.
- Validate alignment with stakeholder requirements and functional specifications by mapping all core and supplementary application features to clearly defined actors and system processes.
- Document design rationale at each architectural layer to explain why specific decisions were made, thereby supporting future maintenance, enhancement, and peer review of the system.

Each subsequent chapter of this report addresses one layer of the system architecture in turn, beginning with the high-level context and progressing through data flow, functional behaviour, object structure, interaction sequences, and physical deployment.

2. Context Diagram

2.1 Definition of a Context Diagram

A Context Diagram — also referred to as a Level 0 Data Flow Diagram — is a high-level graphical model that defines the boundary of a system and illustrates its interactions with the external environment. It deliberately abstracts away all internal processes and data stores, focusing exclusively on what information flows into and out of the system, and from or to which external entities those flows originate or terminate.

Context diagrams are invaluable in the early stages of systems analysis and design because they establish the scope of the system under development. By clearly depicting who or what communicates with the system — without revealing internal complexity — they serve as a foundation for more detailed modelling at subsequent levels of abstraction. Both technical and non-technical stakeholders can interpret a context diagram without requiring specialist knowledge of software design.

2.2 Components of a Context Diagram

A context diagram is composed of three fundamental elements:

Component	Symbol / Notation	Description
System (Central Process)	Circle or rounded rectangle at centre	Represents the entire system being analysed as a single, indivisible process.
External Entities	Rectangles surrounding the process	Represent people, organisations, or other systems that interact with the system by sending or receiving data.
Data Flows	Labelled arrows between entities and the system	Show the direction and nature of information exchanged between the system and each external entity.

2.3 Context Diagram of CheckAR

The CheckAR Context Diagram below illustrates the system as a single central process — the Car Diagnostic Application — surrounded by the seven external entities with which it communicates. Each data flow is labelled to indicate the type of information being exchanged. The diagram confirms that CheckAR receives input from users in the form of dashboard images and engine audio, interfaces with three external API providers, and communicates bidirectionally with an administrator for system governance purposes.

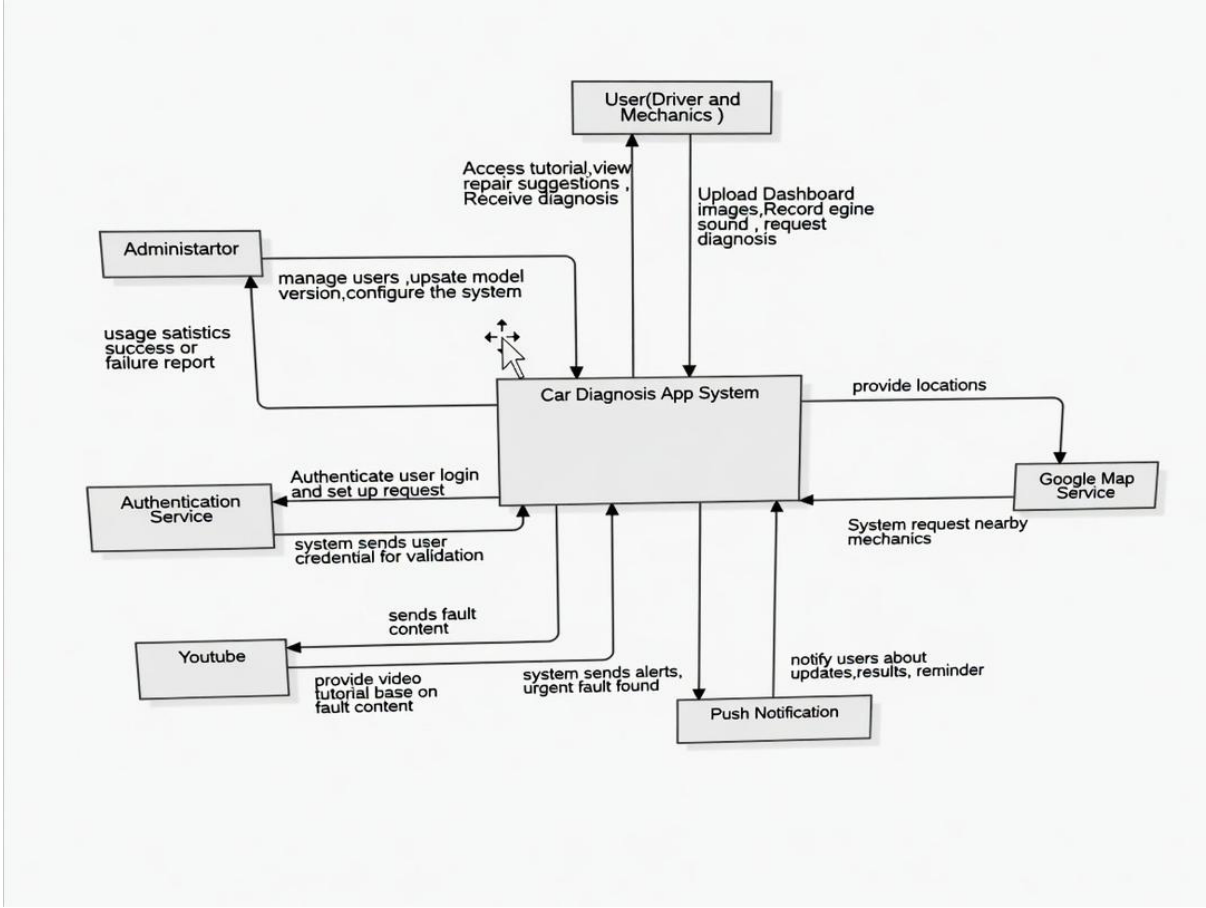

Figure 2.1: Context Diagram of the CheckAR Car Fault Diagnosis Application

2.4 Description of Context Diagram Components

The following subsections describe each component of the CheckAR context diagram in detail, explaining both the nature of the external entity and the specific data flows it shares with the system.

2.4.1 CheckAR System (Central Process)

The CheckAR system is represented as the central process in the diagram. It is the AI-powered mobile and cloud-based backend platform through which all diagnostic activity is coordinated. The system receives multimodal input — images and audio — from users, processes that input using machine learning models, and returns structured diagnostic reports. It also communicates with external services to enrich its functionality and with the administrator to support governance and model management. The system boundary encompasses both the Flutter mobile client and the Node.js/Firebase backend infrastructure.

2.4.2 Administrator

The Administrator is an authorised technical user responsible for the ongoing management and governance of the CheckAR system. The administrator sends system commands to the backend, including instructions for managing registered user accounts, configuring application settings, and deploying updated machine learning models. In return, the system provides the administrator with performance reports, diagnostic accuracy metrics, user activity logs, and system health alerts. The administrator interacts with CheckAR through a dedicated web-based admin dashboard rather than the mobile application.

2.4.3 Car User (Driver / Mechanic)

The Car User is the primary end-user of the CheckAR application. This actor encompasses both vehicle owners who wish to understand warning lights or unusual sounds in their car, and mechanics who may use the app to assist with initial fault assessments. The car user provides the system with input data — specifically, photographs of the vehicle dashboard and short recordings of engine sounds — and may also supply vehicle profile information and location data. In return, the system delivers comprehensive diagnostic reports, urgency-rated maintenance reminders, suggested repair actions, relevant video tutorials, and the locations of nearby mechanics.

2.4.4 Google Maps Service

The Google Maps Service is an external third-party API integrated into CheckAR to support the mechanic location feature. When a user requests information about nearby repair centres or mechanics, the system sends a location query to the Google Maps API. The API responds with geographic data — including the names, distances, ratings, and directions to relevant service providers in the user's vicinity. This integration allows CheckAR to provide actionable, location-aware guidance following a diagnostic event.

2.4.5 Push Notification Service

The Push Notification Service — implemented through Firebase Cloud Messaging (FCM) — enables CheckAR to deliver time-sensitive alerts to users asynchronously. When the system identifies a fault with a critical or high urgency rating, it sends a notification trigger to the FCM service, which in turn delivers a push notification to the user's device. This service also handles scheduled maintenance reminders that the system generates based on the urgency classifications of previously identified faults. The push notification service operates independently of the main application thread, ensuring that alerts are received even when the application is not actively in use.

2.4.6 YouTube Data API

The YouTube Data API is used by CheckAR to retrieve contextually relevant tutorial videos following a diagnostic event. Once the system has identified a specific fault or warning light, it queries the YouTube API with search terms derived from the diagnosis result. The API returns a list of suitable video results — including titles, thumbnails, and playback URLs — which the system displays within the diagnostic report to help users understand the issue and, where appropriate, attempt basic repairs themselves. This integration reduces the need for users to independently search for repair guidance.

2.4.7 Authentication Service

The Authentication Service — provided by Firebase Authentication — manages all aspects of user identity verification within CheckAR. When a user registers or attempts to log in, the mobile client forwards the user's credentials to the authentication service. The service validates those credentials and returns a secure session token that the application uses to authorise subsequent requests. The authentication service also supports email verification during registration and provides the password-reset token flow used in the account recovery process. By delegating authentication to a dedicated service, CheckAR avoids managing sensitive credential data directly within its own backend.

3. Data Flow Diagram (DFD)

3.1 Definition of a Data Flow Diagram

A Data Flow Diagram (DFD) is a structured graphical representation of the movement and transformation of data within a system. Unlike the context diagram, which treats the system as a single process, a DFD decomposes the system into its constituent processes and shows how data flows among those processes, the entities that interact with them, and the data stores in which persistent information is held. DFDs are a cornerstone of structured systems analysis and are used at multiple levels of detail: Level 1 DFDs decompose the single process of the context diagram into its major functional sub-processes, while Level 2 and beyond provide further detail for individual processes.

For CheckAR, the Level 1 DFD presented in this chapter decomposes the system into seven core processes, five data stores, and three categories of external entity. This level of decomposition is sufficient to capture all major information flows and process responsibilities without introducing unnecessary complexity.

3.2 Components of a Data Flow Diagram

A DFD uses four standardised notation elements, each with a specific meaning:

Component	Symbol	Description
Process	Rounded rectangle / circle (numbered)	Represents a function or transformation that receives input data, performs an operation, and produces output data.
Data Store	Open-ended rectangle (labelled DS#)	Represents a repository where data is held at rest — such as a database table or file — for use by one or more processes.
External Entity	Rectangle (labelled with entity name)	Represents a person, organisation, or external system that is the source or destination of data but lies outside the system boundary.
Data Flow	Labelled directed arrow	Represents the movement of a specific type of data between any two components — entities, processes, or data stores.

3.3 Data Flow Diagram of CheckAR

The Level 1 DFD of CheckAR, shown in Figure 3.1 below, illustrates how data moves through the system's seven core processes in response to inputs from three categories of external entity. Each process is numbered (P1 through P7), each data store is labelled (DS1 through DS5), and each data flow arrow is labelled with the type of information it carries. The diagram makes explicit the interdependencies between processes — for example, both the dashboard scanner (P2) and the engine sound analyser (P3) feed their results into the diagnosis and reporting process (P4), which in turn populates the diagnostic history store (DS3) consulted by the maintenance tracking process (P5).

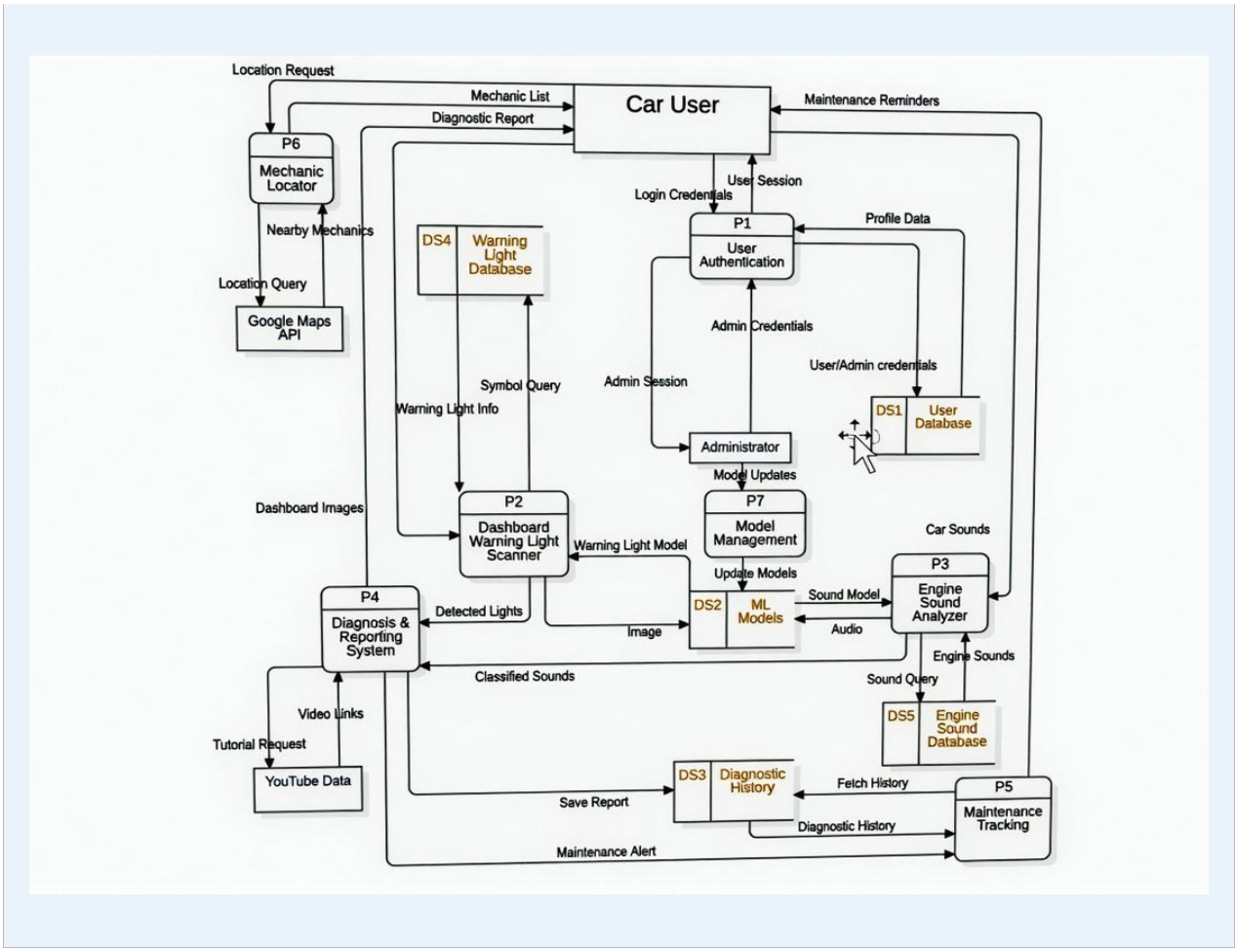

Figure 3.1: Level 1 Data Flow Diagram of the CheckAR Application

3.4 External Entities

3.4.1 Car User

The Car User is the most active external entity in the DFD. This entity supplies the system with login credentials during authentication, dashboard images for warning light scanning, and engine audio recordings for sound analysis. In return, the Car User receives authenticated session tokens, comprehensive diagnostic reports, maintenance reminders and alerts, and map-based mechanic location results. The Car User also interacts with the diagnostic history store indirectly through the maintenance tracking process.

3.4.2 Administrator

The Administrator provides administrative credentials to authenticate with the system and sends model update packages and system configuration commands to the backend. In return, the administrator receives admin access tokens, system performance reports, user management data, and confirmation of successful model deployments. The administrator's primary interaction points are the user authentication process (P1) and the model management process (P7).

3.4.3 External APIs

Three external API services act as specialised external entities within the DFD. The Google Maps API receives location queries from the mechanic locator process (P6) and returns geographic data about nearby repair services. The YouTube Data API receives fault-based

search queries from the diagnosis and reporting process (P4) and returns video content metadata. The Firebase Authentication Service receives credential validation requests from the user authentication process (P1) and returns session tokens and verification responses. These APIs are treated as external entities because their internal operation lies entirely outside the CheckAR system boundary.

3.5 System Processes

3.5.1 P1 – User Authentication

The User Authentication process manages all aspects of user identity within the system. It receives registration data (email address, password, and role selection) and login credentials from both Car Users and Administrators. The process validates credentials against stored user profiles in DS1 and communicates with the Firebase Authentication Service for external verification and token issuance. Upon successful authentication, it creates or updates session records and returns access tokens to the requesting entity. It also handles the password recovery flow, generating and verifying time-limited reset tokens.

3.5.2 P2 – Dashboard Warning Light Scanner

The Dashboard Warning Light Scanner process receives raw dashboard images from authenticated Car Users. It passes these images to the Image Processing sub-module, which performs enhancement operations including contrast normalisation, noise reduction, and resizing. The processed image is then analysed using the machine learning model retrieved from DS2, and identified warning symbols are cross-referenced with the Warning Light Database (DS4) to retrieve their meanings, severity levels, and associated vehicle systems. The process outputs a structured set of warning light identifications — including icon descriptions and urgency ratings — which are forwarded to the Diagnosis and Reporting System (P4).

3.5.3 P3 – Engine Sound Analyser

The Engine Sound Analyser process receives audio recordings submitted by Car Users. It applies a series of audio preprocessing operations — including noise filtering, normalisation, and acoustic feature extraction (e.g., Mel-frequency cepstral coefficients) — before forwarding the feature vectors to the machine learning classification model in DS2. The model classifies the sound against known fault categories (such as knocking, grinding, squealing, hissing, or clicking). Classification results are then augmented with detailed fault descriptions and probable causes retrieved from the Engine Sound Database (DS5), and the combined output is forwarded to P4 for report generation.

3.5.4 P4 – Diagnosis and Reporting System

The Diagnosis and Reporting System is the central integration process of the CheckAR application. It receives classification outputs from both P2 and P3, combines them into a unified diagnostic picture, and generates a comprehensive diagnostic report. The report includes identified faults, their probable causes, urgency classifications, and recommended corrective actions. The process queries the YouTube Data API to retrieve relevant tutorial videos for each identified fault and embeds video links within the report. Completed reports are stored in the Diagnostic History data store (DS3) and returned to the Car User via the mobile application interface.

3.5.5 P5 – Maintenance Tracking

The Maintenance Tracking process manages the longitudinal record of a user's vehicle health over time. It reads diagnostic reports from DS3 and organises them into a chronological maintenance history accessible to the Car User. The process also analyses the urgency ratings of outstanding faults and generates appropriately timed maintenance reminders: faults classified as critical (red) trigger immediate alerts, high-urgency faults (amber) trigger one-

hour reminders, and monitoring-level faults generate two-hour follow-up alerts. Reminder triggers are dispatched to the Push Notification Service for delivery to the user's device.

3.5.6 P6 – Mechanic Locator

The Mechanic Locator process handles location-based service requests initiated by Car Users following a diagnostic event. It receives the user's current location (or a manually entered location if permission is denied) and formulates a structured query for the Google Maps API. The API response — containing a list of nearby mechanics and repair centres with ratings, distances, and contact details — is processed and formatted by P6 before being returned to the user. The process also supports filtering of results by distance, specialisation, and user rating.

3.5.7 P7 – Model Management

The Model Management process is an administrator-facing process responsible for maintaining the integrity and currency of the machine learning models deployed within CheckAR. Administrators upload new model versions through the admin dashboard, and P7 validates those models (checking format, version metadata, and performance benchmarks) before staging them for deployment. Once confirmed by the administrator, the process updates DS2 with the new model files and triggers an over-the-air (OTA) update notification to mobile clients. The process also maintains a version history to support rollback in the event of a degraded model performance.

3.6 Data Stores

3.6.1 DS1 – User Profiles

The User Profiles data store holds all persistent user account information, including unique user identifiers, email addresses, hashed passwords, role assignments (car user or administrator), profile preferences, notification settings, and vehicle profile data. It is the primary data store accessed by the User Authentication process (P1) and is also consulted by the Model Management process (P7) when performing role-based access validation. In the deployed system, this data store is backed by Cloud Firestore, with offline caching provided by SQLite on the mobile device.

3.6.2 DS2 – ML Models

The ML Models data store holds the serialised machine learning model files used by the Dashboard Warning Light Scanner (P2) and the Engine Sound Analyser (P3). Each model is stored with version metadata, performance benchmarks, and compatibility information. On mobile devices, optimised TensorFlow Lite (.tflite) model files are cached locally in this store to support offline inference. The Model Management process (P7) has exclusive write access to this store, ensuring that all model updates are validated before deployment.

3.6.3 DS3 – Diagnostic History

The Diagnostic History data store is a persistent repository of all diagnostic reports generated for a given user. Each record includes a timestamp, the input data type (image, audio, or both), the identified faults, their urgency ratings, recommended actions, and the associated tutorial video links. This store is written to by the Diagnosis and Reporting System (P4) and read by the Maintenance Tracking process (P5). Users can query this store through the application to review their vehicle's diagnostic timeline and monitor recurring issues.

3.6.4 DS4 – Warning Light Database

The Warning Light Database is a structured reference store containing information about all supported dashboard warning light symbols. For each symbol, the database holds a visual descriptor or hash (for matching), the symbol's name, a plain-language explanation of its meaning, the vehicle system it relates to (e.g., engine, brakes, transmission), its urgency

classification, and recommended user actions. This store is read-only during normal operation and is updated only by the administrator when new vehicle makes or symbols need to be added.

3.6.5 DS5 – Engine Sound Database

The Engine Sound Database holds reference profiles for known engine fault sound patterns. Each entry includes a fault category label (e.g., knocking, grinding), a textual description of the sound signature, a list of probable mechanical causes, an urgency rating, and recommended diagnostic or repair actions. Like DS4, this store is maintained by the administrator and is accessed in read mode by the Engine Sound Analyser (P3) to enrich machine learning classification outputs with human-readable fault descriptions.

3.7 Key Data Flows

The following table summarises the principal data flows in the CheckAR DFD, organised by functional category:

Category	Data Flows	Description
Authentication	Credentials → P1; Token ← P1	User and admin credentials validated; session tokens returned for authorised access.
Diagnostic Input	Image → P2; Audio → P3	Dashboard photographs and engine recordings submitted by Car Users for analysis.
Diagnostic Output	Report ← P4	Consolidated fault reports with urgency ratings, causes, and tutorial links returned to users.
Maintenance	History ↔ DS3; Reminders → User	Past reports retrieved for history view; urgency-based reminders dispatched via FCM.
Location	Location → P6; Results ← Google Maps	User location sent to P6; nearby mechanics returned from Google Maps API.
Model Management	Model Package → P7; Update → DS2	New model versions uploaded by admin, validated, stored, and pushed OTA to clients.
External API	Queries → YouTube / Google Maps / Firebase	Search, location, and authentication queries dispatched to external services.

4. Use Case Diagram

4.1 Introduction to Use Case Diagrams

A Use Case Diagram is a type of behavioural Unified Modelling Language (UML) diagram that represents the functional requirements of a system from the perspective of its users and external systems — collectively referred to as actors. Each use case represents a discrete goal that an actor can achieve by interacting with the system, such as submitting an image for analysis, viewing a diagnostic report, or managing user accounts. Use case diagrams do not describe internal system logic; they focus on what the system does rather than how it does it.

The relationship between actors and use cases is expressed through associations (lines connecting an actor to the use cases they participate in), include relationships (indicating that one use case always incorporates the behaviour of another), and extend relationships (indicating that one use case optionally augments another under specific conditions).

4.2 Importance of Use Case Diagrams for CheckAR

Use case diagrams serve several important purposes within the CheckAR project:

1. Stakeholder Communication: The diagrams provide a visual representation of system functionality that is accessible to non-technical stakeholders — including vehicle owners, mechanics, and project sponsors — as well as developers and architects.
2. Scope Definition: By explicitly listing all supported use cases, the diagrams establish clear boundaries for what is in scope (e.g., dashboard light scanning, engine sound analysis) and what is excluded (e.g., direct OBD-II hardware connection, real-time video streaming).
3. Role Clarification: With two distinct user roles — Car User and Administrator — the use case diagram clearly delineates which features are available to each, preventing scope creep and unauthorised access in implementation.
4. Requirements Traceability: Each use case can be traced back to one or more functional requirements in the System Requirements Specification (SRS), providing a verifiable link between user needs and system behaviour.
5. Test Planning: Each use case constitutes a testable unit of behaviour, providing a natural basis for the construction of test cases and acceptance criteria during the quality assurance phase.

4.3 Use Case Diagram of CheckAR

The CheckAR use case diagram below depicts fifteen use cases distributed between two primary actors: the Car User and the System Administrator. The system boundary rectangle encloses all use cases, while actors are positioned outside it. Include and extend relationships are shown where one use case depends upon or optionally augments another.

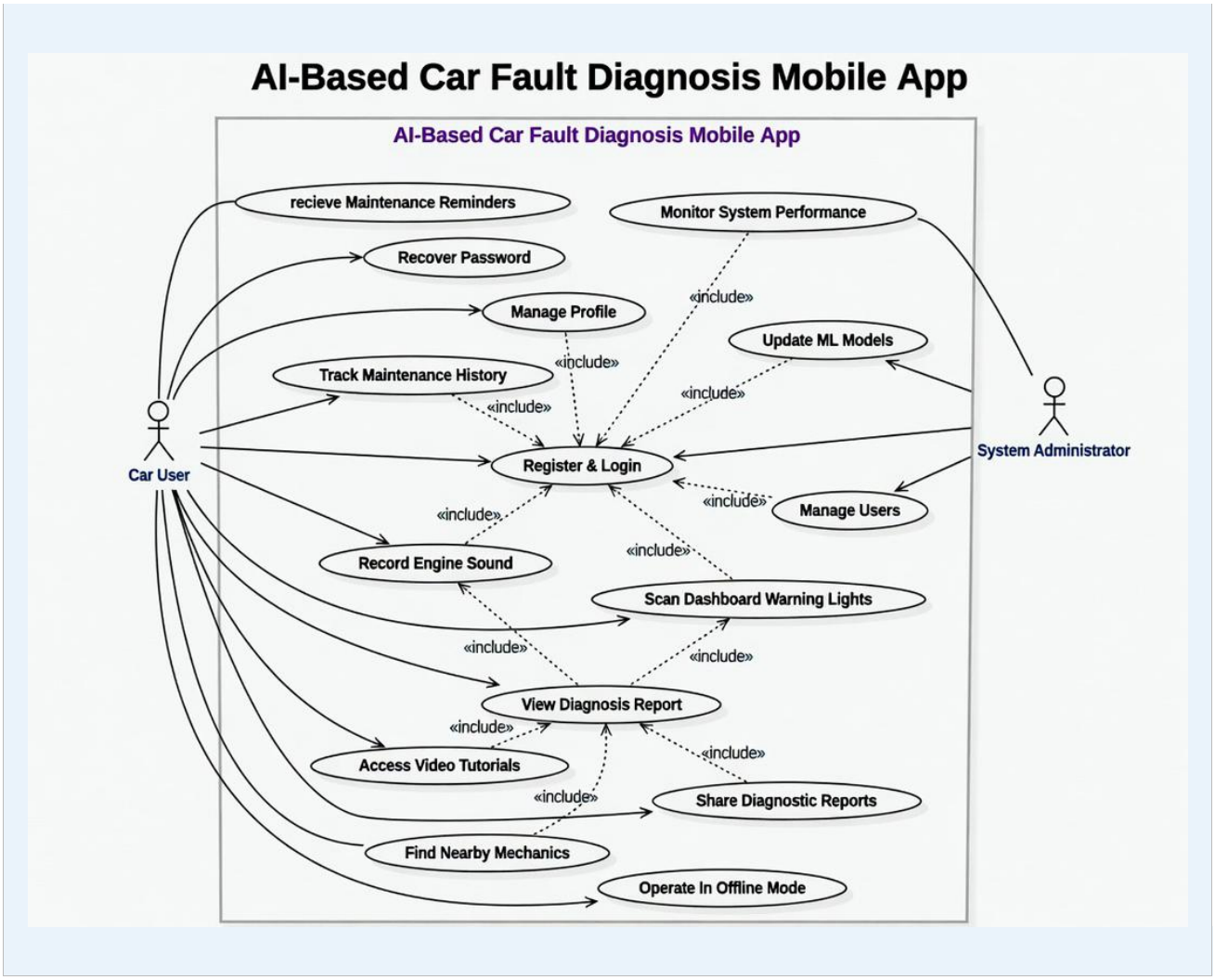

Figure 4.1: Use Case Diagram of the CheckAR Application

4.4 Detailed Use Case Descriptions

The following subsections provide structured descriptions for each of the fifteen use cases identified in the CheckAR use case diagram. Each description follows a standardised template covering the use case identifier, name, participating actors, preconditions, main success flow, alternative flows, postconditions, and the associated functional requirements.

4.4.1 UC1 – Register and Login

Register and Login	
Use Case ID	UC1
Actors	Car User, Administrator
Description	Allows new users to create a CheckAR account by providing an email address, selecting a role, and verifying their email. Returning users may log in with their registered credentials. This use case is the entry point to all other system functionality and governs role-based access control throughout the application.

Preconditions	The CheckAR application is installed on the user's device. The user has a valid email address and internet connectivity.
Main Flow	1. User opens the application and selects 'Register'. 2. User enters email address, password, and selects a role (Car User or Administrator). 3. System validates input format and checks for duplicate accounts. 4. System sends an email verification link to the provided address. 5. User clicks the verification link in their email inbox. 6. System marks the account as verified and prompts the user to complete their profile. 7. User submits profile information (name, vehicle details). 8. System stores the profile in DS1 and grants role-based access. 9. User is redirected to the application home screen.
Alternative Flows	A1 – Returning User Login: User selects 'Login', enters credentials, and receives an authenticated session token. A2 – Forgot Password: User selects 'Forgot Password', enters registered email, receives a reset token, and sets a new password (see UC13). A3 – Duplicate Email: System displays an error message and prompts the user to use a different email or recover their existing account.
Postconditions	The user is authenticated and granted access to features appropriate to their assigned role.
Requirements	FR1, FR2, FR3, FR4

4.4.2 UC2 – Scan Dashboard Warning Lights

Scan Dashboard Warning Lights	
Use Case ID	UC2
Actors	Car User
Description	Enables the Car User to capture a photograph of their vehicle's instrument cluster using the device camera. The image is processed by the machine learning model to identify and interpret any illuminated warning light symbols, providing the user with an explanation of each symbol's meaning and urgency level.
Preconditions	The user is authenticated and has an active session. Camera permission has been granted by the device operating system. The vehicle dashboard is reasonably visible (adequate ambient lighting).
Main Flow	1. User selects 'Scan Dashboard' from the application home screen. 2. The application launches the camera interface with a framing guide overlay. 3. User positions the device to capture the dashboard and taps the capture button. 4. The system transmits the image to the Image Processing Module for preprocessing.

	5. The preprocessed image is analysed by the warning light ML model (from DS2). 6. Identified warning symbols are highlighted on the image and cross-referenced with DS4. 7. The system displays the list of detected symbols with names, explanations, and urgency ratings. 8. User confirms the image is satisfactory; the system proceeds to generate a diagnosis report (UC4).
Alternative Flows	A1 – No Lights Detected: System displays a message confirming no warning lights were identified and suggests improving lighting or camera angle before retrying. A2 – Camera Permission Denied: System explains the requirement and prompts the user to enable camera access in device settings. A3 – Processing Error: System displays an error notification and invites the user to retry with a clearer image.
Postconditions	Dashboard warning lights are identified and their details are passed to the Diagnosis and Reporting System for report generation.
Requirements	FR5, FR6, FR15

4.4.3 UC3 – Record Engine Sound

Record Engine Sound	
Use Case ID	UC3
Actors	Car User
Description	Allows the Car User to record a short audio sample of their engine using the device microphone. The recording is processed by the audio analysis module to extract acoustic features, which are then classified by the machine learning model to identify potential mechanical fault patterns.
Preconditions	The user is authenticated and has an active session. Microphone permission has been granted by the device operating system. The user is in a sufficiently quiet environment to capture a clear engine recording.
Main Flow	1. User selects 'Record Engine Sound' from the application home screen. 2. The system displays recording instructions and guidance on optimal device positioning. 3. User positions the device near the engine bay and taps 'Start Recording'. 4. The system records audio for a duration of 5–10 seconds. 5. User taps 'Stop Recording'; the audio file is transmitted to the Audio Processing Module. 6. The module applies noise reduction, normalisation, and acoustic feature extraction. 7. Extracted features are classified by the engine sound ML model (from DS2). 8. Classification results are cross-referenced with DS5 to retrieve fault descriptions.

	9. The system forwards results to the Diagnosis and Reporting System (UC4).
Alternative Flows	A1 – Microphone Permission Denied: System explains the requirement and directs the user to device permission settings. A2 – Recording Too Noisy: System analyses ambient noise level and advises the user to find a quieter environment before retrying. A3 – Recording Too Short: System requests that the user record for at least 5 seconds to ensure adequate feature extraction.
Postconditions	Engine fault patterns are identified from the audio recording and forwarded to the Diagnosis and Reporting System for report generation.
Requirements	FR7, FR8

4.4.4 UC4 – View Diagnosis Report

View Diagnosis Report	
Use Case ID	UC4
Actors	Car User
Description	Generates and displays a comprehensive diagnostic report by combining the results of dashboard warning light identification (UC2) and/or engine sound analysis (UC3). The report presents identified faults, their probable causes, urgency classifications, recommended corrective actions, and links to relevant video tutorials retrieved from the YouTube Data API.
Preconditions	The user has completed at least one of: a dashboard scan (UC2) or an engine sound recording (UC3), or both.
Main Flow	1. The Diagnosis and Reporting System (P4) combines available classification outputs from UC2 and/or UC3. 2. The system determines each fault's urgency level: Critical (red), High (amber), or Monitoring (blue). 3. The system queries the YouTube Data API for tutorial videos relevant to each identified fault. 4. The system generates a structured diagnostic report containing: – Identified fault names and descriptions – Probable mechanical causes – Urgency ratings with colour-coded indicators – Recommended user actions – Embedded tutorial video links 5. The report is displayed on the user's device within the application. 6. The completed report is automatically saved to the Diagnostic History (DS3).
Alternative Flows	A1 – No Issues Detected: System displays an all-clear confirmation message indicating no faults were identified. A2 – Indeterminate Diagnosis: System acknowledges uncertainty and recommends professional inspection. A3 – Critical Issue Detected: System triggers an immediate alert notification in addition to displaying the report.

Postconditions	The user is fully informed about the identified vehicle issues, their urgency, and recommended next steps. The report is persisted in the diagnostic history for future reference.
Requirements	FR9, FR16, FR20

4.4.5 UC5 – Access Video Tutorials

Access Video Tutorials	
Use Case ID	UC5
Actors	Car User
Description	Provides the Car User with contextually relevant video content linked to the faults identified in their diagnostic report. Tutorial links are retrieved from the YouTube Data API and embedded within the report view, allowing users to watch instructional content that explains the identified issue and guides them through potential remediation steps.
Preconditions	The user has viewed a diagnostic report containing at least one identified fault. An internet connection is available (for live video retrieval); or cached tutorial data is present (for offline access).
Main Flow	1. The user views a completed diagnostic report (UC4). 2. The report displays a 'Tutorials' section with one or more video thumbnails and titles. 3. The user taps a tutorial link. 4. The system loads and plays the selected video within an embedded player or opens it in the YouTube application. 5. The user watches the tutorial and returns to the report.
Alternative Flows	A1 – No Internet Connection: System checks for cached tutorials relevant to the identified fault and displays them if available. If no cache exists, system notifies the user that tutorials are unavailable offline. A2 – No Relevant Tutorials Found: System displays a message and offers a selection of general vehicle maintenance tutorial links as an alternative.
Postconditions	The user gains visual, instructional understanding of the identified fault and its potential resolution.
Requirements	FR10

4.4.6 UC6 – Track Maintenance History

Track Maintenance History	
Use Case ID	UC6
Actors	Car User
Description	Allows the Car User to review a chronological log of all diagnostic reports generated for their vehicle within the CheckAR application.

	The history view enables the user to identify recurring issues, monitor the resolution status of previously flagged faults, and track overall vehicle health over time.
Preconditions	The user is authenticated and has at least one previously generated diagnostic report stored in DS3.
Main Flow	1. User navigates to the 'History' section from the application home screen. 2. The system retrieves all diagnostic records from DS3 associated with the user's account. 3. Records are displayed in reverse chronological order, each showing a date, summary of identified faults, and urgency level. 4. User selects a specific report entry. 5. System displays the full diagnostic report for that entry, including all fault details and tutorial links.
Alternative Flows	A1 – No History Available: System displays an empty state screen with an explanation and a prompt to perform the user's first scan. A2 – Filter by Date: User applies a date range filter; system returns records within the specified period. A3 – Filter by Issue Type: User selects a fault category filter; system returns records matching that category.
Postconditions	The user can review their complete vehicle maintenance history and access the details of any previously generated diagnostic report.
Requirements	FR11

4.4.7 UC7 – Receive Maintenance Reminders

Receive Maintenance Reminders	
Use Case ID	UC7
Actors	Car User
Description	The system automatically generates and delivers maintenance reminders to the Car User based on the urgency classification of faults identified in diagnostic reports. Reminders are delivered as push notifications via Firebase Cloud Messaging and are timed according to the severity of the fault.
Preconditions	The user has granted push notification permissions on their device. At least one diagnostic report with a fault urgency rating has been generated and stored in DS3.
Main Flow	1. The Maintenance Tracking process (P5) analyses the urgency ratings in the most recent diagnostic report. 2. Based on urgency level, the system schedules a reminder: – Critical (Red): Immediate push notification dispatched. – High (Amber): Push notification dispatched after 1 hour. – Monitoring (Blue): Push notification dispatched after 2 hours. 3. The system sends the reminder trigger to the Push Notification Service (Firebase FCM). 4. The FCM service delivers the notification to the user's device. 5. User taps the notification to open the relevant diagnostic report.

Alternative Flows	A1 – User Dismisses Notification: System reschedules the reminder according to the urgency level — immediate and high faults are re-notified after 30 minutes; monitoring faults are re-notified after 4 hours. A2 – User Acknowledges and Acts: System updates the reminder status to 'Acknowledged' and stops resending for that specific fault report.
Postconditions	The user is proactively reminded of outstanding vehicle maintenance needs, with reminder frequency calibrated to fault urgency.
Requirements	FR12

4.4.8 UC8 – Share Diagnostic Reports

Share Diagnostic Reports	
Use Case ID	UC8
Actors	Car User
Description	Enables the Car User to share a completed diagnostic report with a third party — such as a mechanic, insurance provider, or family member — using the device's native sharing capabilities or by exporting the report as a PDF document.
Preconditions	The user has at least one completed diagnostic report. Sharing permissions are available on the device.
Main Flow	1. User views a completed diagnostic report (UC4). 2. User taps the 'Share' action button. 3. The system generates a shareable version of the report. 4. The device's native share sheet is displayed, showing available sharing options (email, messaging applications, etc.). 5. User selects a sharing method and recipient. 6. System transmits the report via the selected method.
Alternative Flows	A1 – Export as PDF: User selects 'Export as PDF'; system generates a formatted PDF document and offers it as a downloadable or shareable file. A2 – Share Directly with Mechanic: System generates a specialised mechanic-friendly report format that includes vehicle information alongside the fault data.
Postconditions	The diagnostic report has been successfully shared with the intended recipient via the chosen method.
Requirements	FR17

4.4.9 UC9 – Find Nearby Mechanics

Find Nearby Mechanics	
Use Case ID	UC9
Actors	Car User

Description	Assists the Car User in locating qualified mechanics or repair centres in their geographic vicinity following a diagnostic event. The feature uses the Google Maps API to surface relevant service providers and displays them on an interactive map with filtering and navigation options.
Preconditions	The user has completed a diagnostic session (UC4). An internet connection is available. Location permission is granted (or the user is prepared to enter their location manually).
Main Flow	1. User selects 'Find Nearby Mechanics' from within the diagnostic report or the application menu. 2. System requests location permission if not already granted. 3. System sends the user's location to the Mechanic Locator process (P6), which queries the Google Maps API. 4. System displays an interactive map with markers indicating nearby mechanics and repair centres. 5. User can filter results by distance, available services, or customer ratings. 6. User selects a mechanic to view full details (name, address, phone number, opening hours). 7. System provides navigation directions to the selected mechanic.
Alternative Flows	A1 – Location Permission Denied: System prompts the user to enter a location manually; the map is then centred on the entered location. A2 – No Mechanics Found: System expands the search radius incrementally until results are found, or informs the user that no results were located within a reasonable radius. A3 – Share Report with Mechanic: User selects the option to share the diagnostic report directly with the selected mechanic via the Share use case (UC8).
Postconditions	The user has identified a suitable nearby mechanic and has the information needed to make contact or navigate to them.
Requirements	FR21, FR22

4.4.10 UC10 – Manage Users

Manage Users	
Use Case ID	UC10
Actors	Administrator
Description	Provides the Administrator with comprehensive user account management capabilities through the admin dashboard. The administrator can view, search, edit, disable, or delete user accounts, and configure role-based permissions across the user base.
Preconditions	The administrator is authenticated with administrator-level credentials. The admin dashboard is accessible via a web browser.
Main Flow	1. Administrator logs into the admin dashboard (UC1 – admin flow). 2. Administrator navigates to the User Management section.

	3. System retrieves and displays a paginated list of all registered user accounts from DS1. 4. Administrator selects a user account to view its details. 5. Administrator performs one or more management actions: edit profile information, change role, disable or re-enable account, or delete account. 6. System validates and applies the changes, updating DS1 accordingly. 7. System logs the administrative action for audit purposes.
Alternative Flows	A1 – Search for Specific User: Administrator enters a name or email; system returns matching accounts. A2 – Filter by Role or Status: Administrator applies role or account status filters to narrow the displayed list. A3 – Reset User Password: Administrator triggers a password reset email for a specific user account.
Postconditions	The targeted user accounts have been updated in accordance with the administrative actions performed. All changes are logged for audit traceability.
Requirements	FR18

4.4.11 UC11 – Update Machine Learning Models

Update Machine Learning Models	
Use Case ID	UC11
Actors	Administrator
Description	Enables the Administrator to deploy updated versions of the machine learning models used for dashboard warning light recognition and engine sound classification. Updates are validated before deployment and distributed to mobile clients via over-the-air (OTA) updates.
Preconditions	The administrator is authenticated with administrator-level credentials. New, validated model versions are prepared and ready for upload. An internet connection is available.
Main Flow	1. Administrator accesses the Model Management section of the admin dashboard. 2. System displays current model versions, accuracy benchmarks, and deployment history. 3. Administrator uploads a new model file (e.g., .tflite for mobile deployment). 4. System validates the model: checks file format, version metadata, and runs automated benchmark tests. 5. Administrator reviews validation results and confirms the deployment schedule. 6. System stages the update in DS2 and initiates OTA distribution to eligible mobile clients. 7. System confirms successful deployment and logs the model version transition.

Alternative Flows	A1 – Rollback to Previous Version: If a deployed model causes performance degradation, the administrator initiates a rollback; system reverts DS2 to the last stable version and re-triggers OTA distribution. A2 – Staged Rollout: Administrator configures the update to be deployed to a percentage of users first for validation before full rollout. A3 – Validation Failure: System rejects the uploaded model, displays a detailed error report, and retains the currently deployed version.
Postconditions	The updated ML models are deployed to DS2 and distributed to mobile clients, improving diagnostic accuracy for all users.
Requirements	FR14

4.4.12 UC12 – Monitor System Performance

Monitor System Performance	
Use Case ID	UC12
Actors	Administrator
Description	Provides the Administrator with a comprehensive analytics and monitoring dashboard that surfaces key performance indicators for the CheckAR system. The administrator can monitor model accuracy, track user engagement patterns, investigate error rates, and export performance reports for management review.
Preconditions	The administrator is authenticated with administrator-level credentials. The system monitoring service is operational.
Main Flow	1. Administrator accesses the System Dashboard from the admin interface. 2. System displays an overview of key performance metrics, including: – Active user count and session trends – ML model accuracy rates (by model type and version) – Diagnostic request volumes and processing times – Error rates and error type breakdowns 3. Administrator selects a specific metric for detailed analysis. 4. System renders detailed charts, time-series graphs, or tabular data for the selected metric. 5. Administrator may export a performance report in PDF or CSV format.
Alternative Flows	A1 – Filter by Date Range: Administrator applies a custom date range; system filters all metrics accordingly. A2 – Configure Alert Thresholds: Administrator sets threshold values for specific metrics (e.g., error rate > 5%); system sends automated alerts when thresholds are exceeded. A3 – Investigate Specific Error: Administrator drills into an error category to view individual error logs and associated diagnostic sessions.

Postconditions	The administrator has a comprehensive, data-driven understanding of system performance and can make informed decisions about maintenance, model updates, or infrastructure scaling.
Requirements	Inferred from administrator role responsibilities; supports FR14 and FR18.

4.4.13 UC13 – Recover Password

Recover Password	
Use Case ID	UC13
Actors	Car User
Description	Provides a secure self-service mechanism for Car Users who have forgotten their password to regain access to their account by requesting a time-limited password reset link delivered to their registered email address.
Preconditions	The user has a registered and verified account in DS1. The user has access to their registered email inbox. An internet connection is available.
Main Flow	1. User selects 'Forgot Password' on the login screen. 2. System prompts the user to enter their registered email address. 3. System validates that the email corresponds to an existing account in DS1. 4. System generates a time-limited, single-use password reset token. 5. System sends a password reset email containing the token (as a link) to the user's registered address. 6. User opens the email and clicks the reset link. 7. System validates the token (checks expiry and single-use status). 8. System presents a password reset form; user enters and confirms a new password. 9. System validates the new password against complexity requirements and updates DS1. 10. System confirms successful reset and redirects the user to the login screen.
Alternative Flows	A1 – Email Not Recognised: System displays a message indicating that no account is associated with the provided email, without revealing whether an account exists (for security). A2 – Token Expired: System informs the user that the reset link has expired and offers to send a new one. A3 – New Password Fails Validation: System identifies the specific rule that was violated (e.g., insufficient length) and prompts the user to correct their entry.
Postconditions	The user's password has been securely updated in DS1. The user can log in with their new credentials.
Requirements	FR3

4.4.14 UC14 – Operate in Offline Mode

Operate in Offline Mode	
Use Case ID	UC14
Actors	Car User
Description	Enables the Car User to access core diagnostic functionality when an internet connection is unavailable. Offline capability is made possible by locally cached ML models (stored as TensorFlow Lite files), a local SQLite database, and pre-downloaded tutorial content. Diagnostic results generated offline are stored locally and synchronised with the cloud when connectivity is restored.
Preconditions	The user has previously authenticated on the device (session token cached locally). The required TensorFlow Lite ML model files have been downloaded to the device. Sufficient local storage is available on the device.
Main Flow	1. The system detects that the device has no active internet connection. 2. System switches to offline mode and notifies the user of limited functionality. 3. User can access the following core features without connectivity: – Scan Dashboard Warning Lights (using locally cached model from DS2-local) – Record and Analyse Engine Sounds (using locally cached model) – View basic diagnostic reports (generated from local inference) – Browse cached tutorial content – Review previously stored diagnostic history (from local SQLite) 4. Diagnostic results are stored in the local SQLite database. 5. When internet connectivity is restored, the system automatically synchronises locally stored results with Cloud Firestore.
Alternative Flows	A1 – ML Models Not Downloaded: System identifies that required model files are not available locally and displays a warning. Full offline diagnosis is unavailable; user is prompted to connect to Wi-Fi to download models. A2 – Sync Conflict on Reconnection: If a conflict is detected between local and cloud records during sync, system retains the most recent entry and logs the conflict for review.
Postconditions	The user has completed a diagnostic session without internet connectivity. Results are stored locally and will be synchronised with the cloud upon reconnection.
Requirements	FR13

4.4.15 UC15 – Manage Profile

Manage Profile	
Use Case ID	UC15
Actors	Car User, Administrator

Description	Allows authenticated users to view and update their personal profile information, including account details, vehicle information, notification preferences, and data usage settings. Administrators may also manage their own administrative profile through this use case.
Preconditions	The user is authenticated and has an active session.
Main Flow	1. User navigates to the 'Profile' section from the application menu. 2. System retrieves and displays the current profile information from DS1. 3. User modifies one or more fields: personal details, vehicle make/model/year, notification preferences, or data usage settings. 4. System validates the updated data (format, completeness, and business rules). 5. System saves the updated profile to DS1 and confirms the changes to the user.
Alternative Flows	A1 – Change Password: User selects 'Change Password', enters current password for verification, and provides a new password. System validates and updates the credential in DS1. A2 – Update Notification Preferences: User toggles specific notification types (e.g., maintenance reminders, critical alerts) on or off; system updates preferences in DS1. A3 – Delete Account: User requests account deletion; system prompts for confirmation, anonymises diagnostic history records (where required for data retention compliance), and removes the account from DS1.
Postconditions	The user's profile in DS1 reflects all changes submitted during the session.
Requirements	FR19

4.5 Relationships Between Use Cases

The CheckAR use case diagram incorporates several important relationships that reflect the dependencies and optional extensions between use cases:

Include Relationships: An include relationship indicates that one use case unconditionally incorporates the behaviour of another. In CheckAR, the following include relationships exist:

- 'View Diagnosis Report' (UC4) includes 'Scan Dashboard Warning Lights' (UC2) and/or 'Record Engine Sound' (UC3), because a diagnosis report cannot be generated without at least one of these diagnostic inputs.
- 'Share Diagnostic Reports' (UC8) includes 'View Diagnosis Report' (UC4), because the report must exist and be viewable before it can be shared.
- 'Find Nearby Mechanics' (UC9) includes 'View Diagnosis Report' (UC4), because the mechanic search is contextually triggered by a completed diagnosis.
- 'Receive Maintenance Reminders' (UC7) includes 'View Diagnosis Report' (UC4), because reminders are generated from the urgency data within a diagnostic report.

Extend Relationships: An extend relationship indicates that one use case optionally augments another under specific conditions. In CheckAR, the following extend relationship exists:

- 'Recover Password' (UC13) extends 'Register and Login' (UC1), as it is invoked only when the user explicitly selects the forgotten password option during the login flow.

4.6 Use Case Access by Actor

The following table summarises the use cases accessible to each actor in the CheckAR system:

Actor	Accessible Use Cases	Role Summary
Car User	UC1, UC2, UC3, UC4, UC5, UC6, UC7, UC8, UC9, UC13, UC14, UC15	Primary end-user of diagnostic features, maintenance tracking, and vehicle services.
Administrator	UC1, UC10, UC11, UC12, UC15	Backend system manager responsible for user governance, model lifecycle, and performance oversight.

4.7 Technical Implementation Considerations

The use case diagram directly informs several key architectural and implementation decisions:

6. Component Boundaries: The natural groupings of use cases suggest clear component boundaries — an Authentication Module (UC1, UC13), a Diagnostics Module (UC2, UC3, UC4), a Maintenance Module (UC6, UC7), a Social/Sharing Module (UC8, UC9), and an Administration Module (UC10, UC11, UC12).
7. Data Flow Implications: The include relationships reveal the sequential data flow between components, informing the event-driven architecture between the diagnostic input modules, the reporting engine, and the maintenance tracker.
8. Offline Scope: UC14 confirms that the Diagnostics Module (UC2, UC3, UC4) and the History Module (UC6) must function without a network connection, which dictates the requirement for local model caching and SQLite storage in the mobile application.
9. Test Coverage: Each use case, including its alternative flows, maps directly to test scenarios, providing a complete basis for functional testing and user acceptance testing (UAT).

5. Class Diagram Architecture

5.1 Introduction to the Class Diagram

A Class Diagram is a structural UML diagram that models the static architecture of a software system by depicting its classes, the attributes and operations those classes expose, and the relationships between them. For an object-oriented system like CheckAR — built using Dart/Flutter on the client side and Node.js on the backend — the class diagram provides the definitive blueprint for how domain concepts are translated into code-level constructs.

The CheckAR class diagram maps the system's core domain across four functional modules: User and Authentication Management, Vehicle Management, AI Diagnosis Core, and Notification and System Utilities. Each module groups logically related classes that collaborate to fulfil a subset of the system's functional requirements. The diagram bridges the mobile client (Flutter UI, local AI inference, and SQLite storage) with the Firebase cloud layer and the Node.js backend infrastructure described in the Deployment Architecture (Chapter 7).

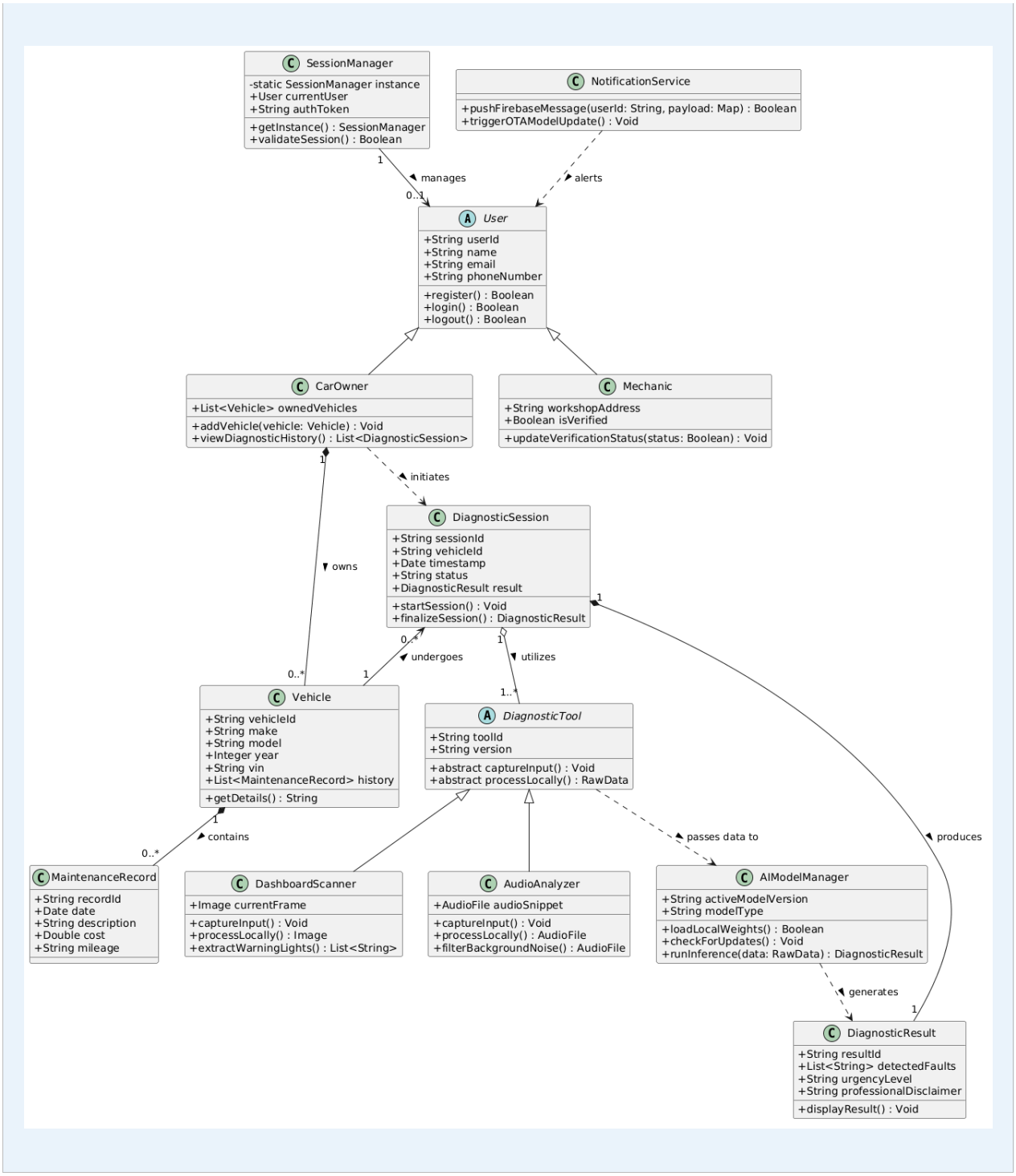

Figure 5.1: Class Diagram of the CheckAR Application

5.2 Core Domain Modules

5.2.1 User and Authentication Management

This module encompasses all classes related to user identity, role management, and session handling. The central class is User, which holds the core account attributes: userId, email, passwordHash, role (an enumeration of 'CarOwner' and 'Mechanic' for regular users, and 'Administrator' for system managers), createdAt, and isVerified. The User class is the base from which two specialised subclasses are derived via inheritance:

- CarOwner: Extends User with vehicle-specific attributes (vehicleId, licensePlate, preferredMechanic) and methods for initiating diagnostic sessions (startDiagnosticSession()) and accessing maintenance history (getMaintenanceHistory()).
- Mechanic: Extends User with professional attributes (licenceNumber, specialisations, workshopLocation) and methods for retrieving assigned diagnostic jobs (getAssignedDiagnostics()).

The SessionManager class is responsible for creating, validating, and terminating authenticated sessions. It holds a reference to the currently authenticated User object, manages the Firebase-issued JWT token (authToken), and exposes methods for login (login()), logout (logout()), and token refresh (refreshToken()). The SessionManager interacts with the Firebase Authentication Service at runtime to validate credentials and issue tokens.

5.2.2 Vehicle Management

This module models the relationship between a user and their vehicle(s). The Vehicle class holds the core attributes of a registered vehicle: vehicleId, make, model, year, registrationNumber, and ownerId (a foreign key reference to the owning CarOwner). It exposes methods for retrieving the vehicle's full diagnostic history (getDiagnosticHistory()) and adding new maintenance records (addMaintenanceRecord()).

The MaintenanceRecord class captures individual maintenance events and is associated with Vehicle through a composition relationship — a maintenance record cannot exist independently of a vehicle. Each record stores a recordId, vehicleId, date, faultDescription, actionTaken, cost, and mechanicId (optionally linking to a Mechanic who performed the service). The MaintenanceRecord class provides methods to mark a record as resolved (markResolved()) and to generate a summary for display in the maintenance history view (getSummary()).

5.2.3 AI Diagnosis Core

The AI Diagnosis Core is the most functionally dense module in the class diagram. It models the end-to-end workflow from diagnostic input to report generation:

- DiagnosticSession: Serves as the orchestrating class for a single diagnostic event. It holds a sessionId, userId, vehicleId, timestamp, and lists of scan and audio results. It exposes methods to initiate a dashboard scan (startDashboardScan()), initiate an audio recording (startAudioRecording()), generate the combined report (generateReport()), and save the session to the diagnostic history (saveToHistory()). DiagnosticSession aggregates both DashboardScanner and AudioAnalyzer.
- DashboardScanner: Responsible for image capture and warning light recognition. Attributes include capturedImage, processedImage, and detectedLights (a list of WarningLight objects). Methods include captureImage(), preprocessImage(), and analyseWarningLights(), the last of which calls the AIModelManager to run inference.
- AudioAnalyzer: Responsible for audio recording and engine fault classification. Attributes include rawAudio, processedFeatures, and detectedFaults (a list of FaultPattern objects). Methods include startRecording(), preprocessAudio(), extractFeatures(), and classifyFault(), again delegating to the AIModelManager.
- AIModelManager (abstract): An abstract class that defines the interface for all machine learning model operations — loadModel(), runInference(), and updateModel(). Concrete implementations include TFLiteModelManager (for on-device TensorFlow Lite inference) and CloudModelManager (for cloud-assisted inference via the backend API). This abstraction allows the system to switch between on-device and cloud inference transparently.
- DiagnosticReport: Holds the consolidated output of a diagnostic session — reportId, sessionId, identifiedFaults (list), urgencyLevel, recommendations, tutorialLinks, and

createdAt. Exposes methods to generate a shareable PDF (generatePDF()) and to format the report for display (displayReport()).

5.2.4 Notification and System Utilities

The NotificationService class handles all push notification logic within the application. It holds a reference to the user's FCM registration token and exposes methods to send an immediate notification (sendImmediateAlert()), schedule a deferred reminder (scheduleReminder(urgencyLevel, delayMinutes)), and cancel a scheduled reminder (cancelReminder(reminderId)). The NotificationService is called by the Maintenance Tracking process whenever a new diagnostic report is saved with an urgency-rated fault.

Supporting utility classes include LocationService (wrapping the Google Maps API integration for the mechanic locator feature), VideoService (wrapping the YouTube Data API queries for tutorial retrieval), and SyncManager (responsible for detecting connectivity changes and triggering local-to-cloud data synchronisation when offline mode transitions back to online).

5.3 Relationships Between Classes

The CheckAR class diagram employs the following UML relationship types:

Relationship Type	Example in CheckAR	Meaning
Inheritance (Generalisation)	CarOwner and Mechanic extend User	Subclasses inherit all attributes and methods of the parent class and add specialised behaviour.
Association	DiagnosticSession references Vehicle	A loosely coupled link between two classes that can exist independently.
Aggregation	DiagnosticSession aggregates DashboardScanner	The whole (session) references the part (scanner), but both can exist independently.
Composition	Vehicle composes MaintenanceRecord	The whole (vehicle) owns the part (record); the record cannot exist without its parent vehicle.
Dependency	AudioAnalyzer depends on AIModelManager	One class uses another transiently (e.g., as a method parameter or return type).
Realisation	TFLiteModelManager realises AIModelManager	A concrete class implements the interface/abstract contract defined by another class.

5.4 Design Rationale

Several deliberate design decisions are reflected in the class diagram structure:

10. Separation of Input and Analysis: DashboardScanner and AudioAnalyzer are modelled as separate classes rather than combined into a single diagnostic class. This separation ensures that each input type can evolve independently — for example, adding support for OBD-II data as a third input type would require adding a new class rather than modifying an existing one.
11. Abstract AI Interface: The AIModelManager abstraction decouples the diagnostic logic from the specific inference engine used. This design supports the system's offline capability (TFLite on-device) and cloud-assisted accuracy improvements (CloudModelManager) without requiring changes to the calling classes.

12. Role-Based Specialisation through Inheritance: Deriving CarOwner and Mechanic from a common User base class avoids duplication of shared attributes (email, password, session management) while allowing each role to expose only the methods relevant to its responsibilities.
13. Composition for Data Integrity: Using a composition relationship between Vehicle and MaintenanceRecord enforces data integrity at the model level — maintenance records are owned by and will be deleted with their parent vehicle, preventing orphaned records in the database.

6. Sequence Diagram Architecture

6.1 Overview

Sequence Diagrams are interaction-level UML diagrams that model the chronological flow of messages between system participants — objects, components, or external actors — in response to a specific scenario or use case. Unlike use case diagrams, which describe what the system does, sequence diagrams describe how it does it, revealing the precise order of method calls, data transmissions, and responses that take place during a given system interaction.

For CheckAR, two sequence diagrams have been developed to document the most technically significant workflows: the Engine Sound Analysis sequence and the Dashboard Warning Light Scan and Analysis sequence. These two workflows represent the core diagnostic capabilities of the application and involve the greatest number of interacting components, making them the most valuable to document at this level of detail.

Each sequence diagram uses a set of lifelines representing the key participants: the Car User, the Mobile Application, the relevant processing module, the Machine Learning Classification Engine, the appropriate fault database, and the Diagnostic Reporting Module. Messages are shown as horizontal arrows between lifelines, with synchronous calls represented by solid arrows and asynchronous responses by dashed return arrows.

6.2 Engine Sound Analysis Sequence Diagram

6.2.1 Diagram Description

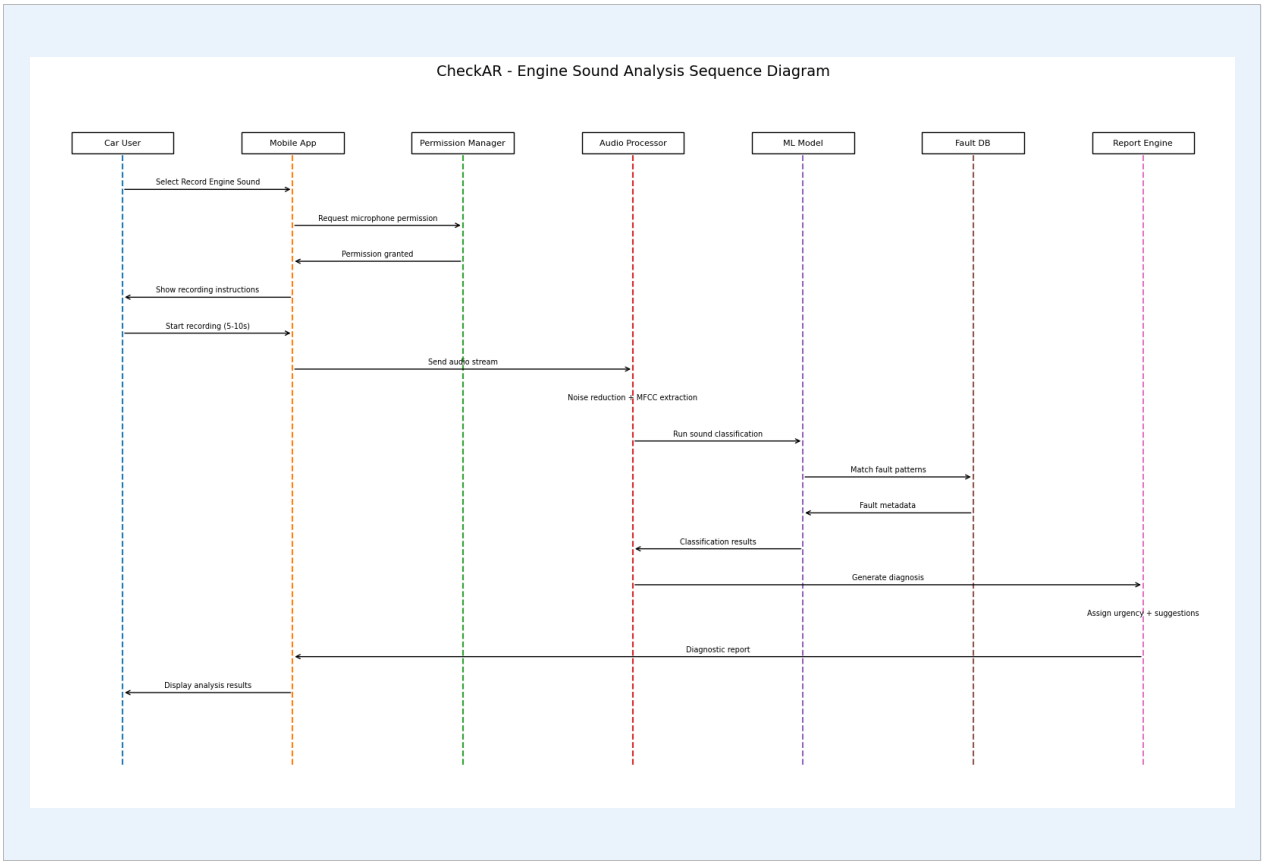

Figure 6.1: Sequence Diagram for the Engine Sound Analysis Workflow

The Engine Sound Analysis Sequence Diagram illustrates the ordered interactions between six participants: the Car User, the Mobile Application, the Audio Processing Module, the ML Classification Engine, the Engine Fault Database (DS5), and the Diagnostic Reporting Module.

The sequence unfolds as follows:

14. Initiation: The Car User selects 'Record Engine Sound' from the application home screen. The Mobile Application sends a permission request to the device operating system for microphone access.
15. Permission Handling: The device grants microphone access (or returns a denial, which triggers the alternative flow described in UC3). The application displays recording instructions and a positioning guide to the user.
16. Audio Capture: The user initiates the recording. The Mobile Application captures a 5–10 second audio clip and forwards it to the Audio Processing Module upon completion.
17. Preprocessing: The Audio Processing Module applies noise reduction, amplitude normalisation, and acoustic feature extraction (computing feature vectors such as Mel-frequency cepstral coefficients). The extracted feature vectors are returned to the application layer.
18. Classification: The feature vectors are forwarded to the ML Classification Engine, which runs inference using the on-device TensorFlow Lite model. The engine produces a classified fault label (e.g., 'Knocking – likely connecting rod bearing') along with a confidence score.
19. Database Enrichment: The Classification Engine queries the Engine Fault Database (DS5) using the fault label as a key. The database returns a full fault record containing: the fault name, a detailed description of the acoustic signature, a list of probable mechanical causes, the urgency rating, and recommended diagnostic or repair actions.
20. Report Generation: The Diagnostic Reporting Module receives the enriched fault data and assembles a structured diagnostic report. The report is returned to the Mobile Application and displayed to the user. Simultaneously, the report is persisted to the Diagnostic History (DS3) and the urgency level is evaluated to determine whether a maintenance reminder should be scheduled.

6.2.2 Design Rationale

The separation of audio preprocessing from machine learning classification is a deliberate design decision. Audio quality varies significantly depending on the user's environment, device microphone quality, and distance from the engine. By isolating preprocessing as an independent module, the quality enhancement pipeline can be tuned, replaced, or extended (for example, to add frequency domain filtering or background noise profiling) without any modification to the classification engine or its training data.

Maintaining the Engine Fault Database as a separate data store, rather than embedding fault descriptions within the ML model itself, provides two important advantages. First, fault descriptions, urgency ratings, and repair recommendations can be updated by the administrator without requiring model retraining — a process that is computationally expensive and time-consuming. Second, the database can be extended to support new fault categories simply by adding new records, without affecting the model's existing classification logic.

The on-device inference architecture (TensorFlow Lite) is reflected in this sequence by the absence of a network call during the classification step. This ensures that the engine sound analysis workflow functions correctly in offline mode (UC14), a critical non-functional requirement for users in areas with unreliable mobile connectivity.

6.3 Dashboard Warning Light Scan and Analysis Sequence Diagram

6.3.1 Diagram Description

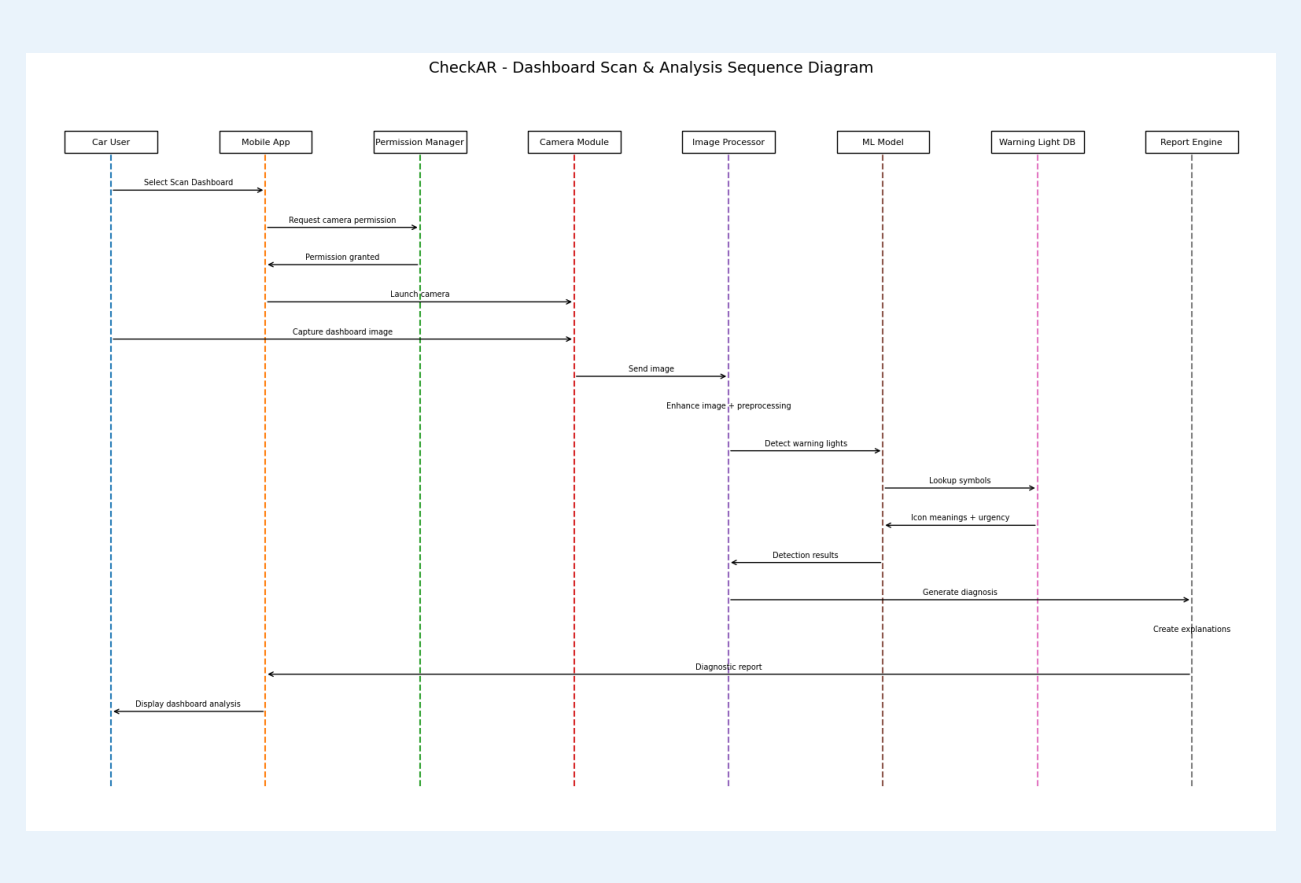

Figure 6.2: Sequence Diagram for the Dashboard Warning Light Scan and Analysis Workflow

The Dashboard Warning Light Scan and Analysis Sequence Diagram documents the ordered interactions between the Car User, the Mobile Application, the Image Processing Module, the ML Classification Engine, the Warning Light Database (DS4), and the Diagnostic Reporting Module.

The sequence unfolds as follows:

21. Initiation: The Car User selects 'Scan Dashboard' from the home screen. The Mobile Application requests camera permission from the device operating system.
22. Camera Launch: The device grants camera access. The application launches the camera interface with a framing guide overlay to help the user correctly position the device to capture the full instrument cluster.
23. Image Capture: The user taps the capture button. The raw image is transmitted from the Mobile Application to the Image Processing Module.
24. Image Preprocessing: The Image Processing Module applies a series of enhancement operations — including histogram equalisation for contrast improvement, Gaussian blur for noise reduction, geometric normalisation for size standardisation, and colour channel separation — to produce a preprocessed image optimised for symbol recognition. The processed image is returned to the application layer.
25. Warning Light Detection and Classification: The preprocessed image is forwarded to the ML Classification Engine, which uses the dashboard warning light recognition model to detect and classify warning symbols present in the image. For each detected symbol, the engine returns a symbol identifier and a confidence score.

26. Database Lookup: The Classification Engine queries the Warning Light Database (DS4) for each identified symbol identifier. The database returns the symbol name, a plain-language explanation, the vehicle system it affects, the urgency classification, and the recommended user action.
27. Report Assembly and Display: The Diagnostic Reporting Module assembles the enriched symbol data into a structured diagnostic report. The detected symbols are highlighted on the original captured image for visual confirmation. The complete report is displayed in the Mobile Application, saved to Diagnostic History (DS3), and evaluated for maintenance reminder generation.

6.3.2 Design Rationale

Image preprocessing is architecturally isolated from the classification model because dashboard images are captured under highly variable real-world conditions: ambient light levels, dashboard reflection, camera angle, and device camera quality all affect raw image quality. A dedicated preprocessing module that can be independently updated ensures consistent input quality to the ML model regardless of capture conditions, thereby maintaining diagnostic accuracy across diverse user devices and environments.

The Warning Light Database is maintained as a separate component — rather than hardcoding symbol descriptions within the model — for the same reasons articulated for the Engine Fault Database in Section 6.2.2. This architecture is particularly valuable for the dashboard scanner because automotive warning light standards evolve regularly as new vehicle systems (e.g., advanced driver assistance systems) are introduced, and new vehicle makes and models are released with proprietary symbols. The database-driven approach allows these updates to be applied by the administrator without any model retraining.

The visual overlay feature — highlighting detected symbols on the original captured image — is implemented in the Mobile Application layer rather than the Image Processing Module to maintain a clean separation of concerns. The processing module is responsible only for feature extraction; the presentation logic for overlaying results on the user's captured image resides in the UI layer of the Flutter application.

7. Deployment Architecture

7.1 Introduction

The Deployment Diagram for CheckAR illustrates the physical and logical distribution of the system's software components across hardware nodes and cloud infrastructure. In UML terminology, a deployment diagram shows which artefacts (executable software components) are deployed on which nodes (hardware or cloud execution environments), and how those nodes communicate with each other. For CheckAR, the deployment architecture reflects the need to balance on-device performance (for offline capability and low latency), cloud scalability (for synchronisation, model updates, and administrative functions), and secure external service integration.

The architecture is composed of five principal tiers: the Mobile Application Layer (running on the user's smartphone), the Backend Application Server (a Node.js/Express service hosted in the cloud), the Firebase Cloud Services Layer (providing authentication, storage, and push messaging), the ML Model Update Server (managing machine learning model versioning and OTA distribution), and the External Service Layer (YouTube Data API and Email Gateway). These tiers communicate through well-defined, secure protocols: HTTPS/REST for synchronous interactions and Firebase Cloud Messaging for asynchronous notifications.

7.2 Deployment Diagram

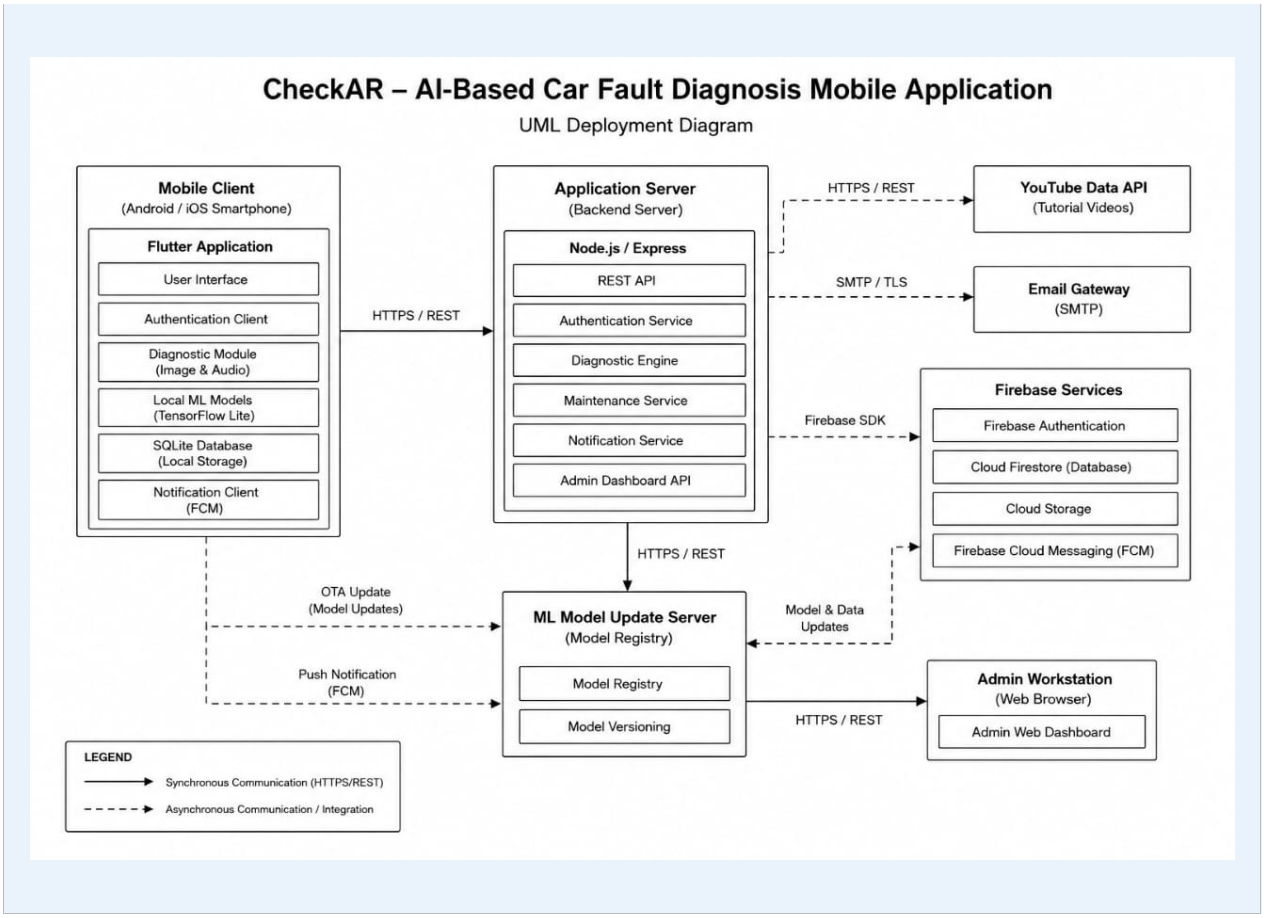

Figure 7.1: Deployment Architecture Diagram of the CheckAR System

The deployment diagram above illustrates all five tiers and their communication pathways. Solid arrows represent synchronous communication (HTTPS/REST), while dashed arrows

represent asynchronous communication (push notifications, OTA model update triggers). The Admin Workstation is shown as a separate node that connects to the Backend Application Server via a web browser, representing the administrator's access to the admin dashboard.

7.3 Mobile Application Layer

The Mobile Application Layer is hosted on the user's Android or iOS smartphone and represents the primary point of interaction for the Car User. It is the most complex tier in the deployment architecture because it must function both online and offline, support real-time AI inference, and manage local data persistence.

The following components are deployed within the Mobile Application Layer:

- User Interface (Flutter): The Flutter-built UI layer renders all screens, handles user interactions, and orchestrates calls to other local components. Flutter's widget-based architecture ensures a consistent, responsive interface across Android and iOS platforms from a single codebase.
- Authentication Client: Manages the local caching of session tokens received from Firebase Authentication and handles the token refresh flow. It communicates with the backend server using Bearer token authorisation on all protected API endpoints.
- Diagnostic Module: Hosts the image capture, audio recording, and local preprocessing logic. This module interfaces directly with device hardware (camera and microphone) and coordinates with the local ML model for on-device inference.
- TensorFlow Lite Models (Local Cache): Serialised .tflite model files for both dashboard warning light recognition and engine sound classification are stored locally on the device. These files are downloaded during initial setup and updated via the OTA mechanism when new versions are available from the ML Model Update Server.
- SQLite Local Database: A lightweight relational database that stores diagnostic reports, maintenance history records, and user profile data locally. This store enables offline access to previously generated diagnostics and serves as a write buffer for results generated in offline mode pending cloud synchronisation.
- Firebase Cloud Messaging (FCM) Client: The notification client registers the device with FCM upon first authentication and listens for incoming push notifications (maintenance reminders, critical fault alerts, model update notifications) delivered by the backend server via the FCM service.

The Mobile Application communicates with the Backend Application Server via HTTPS/REST APIs for all cloud-dependent operations (report synchronisation, mechanic location queries, YouTube tutorial retrieval). All API calls are authenticated using JWT tokens managed by the Authentication Client.

7.4 Backend Infrastructure

The Backend Application Server is a Node.js application built with the Express framework, deployed on a cloud hosting environment (such as Google Cloud Run or similar containerised platform). It serves as the central orchestration hub of the CheckAR system — receiving requests from mobile clients, coordinating with Firebase services, and managing the administrative functions of the application.

The following service components are deployed on the Application Server:

- REST API Gateway: The entry point for all client requests. It handles request routing, input validation, authentication middleware (JWT verification), and rate limiting. All endpoints are served exclusively over HTTPS to protect data in transit.
- Authentication Service: Delegates user credential validation to Firebase Authentication and issues application-level session tokens. Manages role-based access control (RBAC) by embedding role claims in JWT tokens.

- Diagnostic Engine: Coordinates server-side diagnostic processing for cases where cloud inference is requested (e.g., when the device does not have a locally cached model or when the user explicitly requests higher-accuracy cloud analysis). Calls the ML Model Update Server for inference when required.
- Maintenance Service: Manages the generation and scheduling of maintenance reminders based on fault urgency data received from the Diagnostic Engine. Interfaces with Firebase FCM to dispatch notifications.
- Admin Dashboard API: Exposes the backend endpoints consumed by the Administrator's web-based admin dashboard, including user management, model deployment, and performance analytics endpoints.

7.5 Machine Learning Infrastructure

The ML Model Update Server is a dedicated microservice responsible for the lifecycle management of the machine learning models deployed within CheckAR. It operates independently from the main Application Server to ensure that model update operations do not affect the availability or performance of the core diagnostic service.

The ML Model Update Server hosts two key components:

- Model Registry: A versioned repository of all ML model artefacts, storing both the full-precision models (for cloud inference) and the TensorFlow Lite quantised models (for mobile deployment). Each model version is associated with performance benchmark metadata, including accuracy, precision, recall, and F1-score on the diagnostic test dataset.
- Model Versioning and OTA Distribution: Manages the deployment workflow — from administrator upload and automated validation through to staged rollout and OTA push to mobile devices. When a new model version is confirmed for release, the server sends an update notification to the Mobile Application via the Backend Server. The mobile client downloads the new .tflite file over HTTPS and replaces the cached model file, taking effect from the next diagnostic session.

7.6 External Service Integration

CheckAR integrates with two categories of external service:

Firebase Cloud Services

Firebase provides four cloud services that are tightly integrated into the CheckAR architecture. Firebase Authentication handles user credential management and token issuance, eliminating the need for CheckAR to manage password hashing and storage internally. Cloud Firestore is the primary cloud database for user profiles, diagnostic reports, and maintenance records, offering real-time synchronisation capabilities that enable seamless offline-to-online data sync. Cloud Storage hosts larger binary assets, including uploaded dashboard images and audio recordings retained for model retraining purposes. Firebase Cloud Messaging (FCM) provides the push notification infrastructure used by the Maintenance Service to deliver maintenance reminders and critical fault alerts to mobile devices.

YouTube Data API and Email Gateway

The YouTube Data API is called by the Diagnostic Engine on the Backend Server whenever a diagnostic report is generated, using the identified fault labels as search queries. The API returns video metadata (title, description, thumbnail, and playback URL), which is embedded in the diagnostic report and returned to the mobile client. The Email Gateway (an SMTP service such as SendGrid) is used by the Authentication Service to deliver email verification links during registration and password reset tokens during the account recovery flow. All email communications are transmitted over TLS-encrypted SMTP connections.

7.7 Deployment Design Rationale

The deployment architecture reflects several deliberate design decisions aligned with the system's non-functional requirements:

28. On-Device Inference for Offline Support and Low Latency: By deploying TensorFlow Lite models directly on the mobile device, CheckAR can deliver diagnostic results in approximately 1–2 seconds without requiring a network round-trip. This design choice directly satisfies the offline operation requirement (UC14) and ensures acceptable performance for users in low-connectivity environments.
29. Cloud Services for Scalability and Reliability: Delegating authentication, data storage, and push messaging to Firebase cloud services leverages Google's globally distributed infrastructure, providing high availability, automatic scaling, and built-in disaster recovery without requiring custom infrastructure management.
30. Separation of ML Infrastructure: Hosting the ML Model Update Server as an independent service prevents model deployment operations from impacting the availability of the production diagnostic API. This separation also supports the staged rollout and A/B testing capabilities described in UC11.
31. HTTPS Everywhere: All inter-tier communication uses HTTPS, ensuring that user credentials, diagnostic data, and session tokens are protected from interception in transit. Firebase Authentication tokens provide an additional layer of authorisation validation at the Application Server boundary.
32. Microservice-Oriented Backend: The partitioning of the backend into distinct services (Authentication, Diagnostics, Maintenance, Admin, and ML Management) follows a microservice-oriented design philosophy that improves maintainability, enables independent scaling of individual services, and supports future extension of the system with additional diagnostic capabilities or third-party integrations.

8. Conclusion

The CheckAR AI-Based Car Fault Diagnosis Mobile Application represents a thoughtfully engineered response to the challenges that modern vehicle complexity poses for ordinary drivers and mechanics. By combining on-device machine learning inference, cloud-based data management, and seamless third-party service integration within a user-friendly mobile interface, the system delivers intelligent, accessible, and actionable vehicle diagnostics to users regardless of their technical background or proximity to a professional repair centre.

This report has presented a comprehensive, multi-layer architectural analysis of CheckAR through six principal diagram types: Context Diagram, Data Flow Diagram, Use Case Diagram, Class Diagram, Sequence Diagrams, and Deployment Diagram. Together, these diagrams form a complete architectural specification that progresses logically from the highest level of abstraction — the system's external boundaries and interactions — through functional behaviour, information flow, and object-level structure, down to the physical deployment of software components across cloud and device infrastructure.

The Context Diagram established the system's scope by identifying the seven external entities that interact with CheckAR and the nature of the data exchanged between them. The Data Flow Diagram decomposed the system into seven core processes and five data stores, revealing the pathways through which user inputs are transformed into diagnostic outputs and how those outputs are stored, communicated, and acted upon. The Use Case Diagram documented fifteen distinct functional scenarios, clearly delineating the responsibilities of the Car User and Administrator actors and providing a traceable mapping to the system's functional requirements.

The Class Diagram provided the object-oriented structural foundation for implementation, organising the system's domain concepts into four well-defined modules — User and Authentication Management, Vehicle Management, AI Diagnosis Core, and Notification and System Utilities — and establishing clear relationships between classes through inheritance, association, aggregation, and composition. The Sequence Diagrams detailed the chronological interaction flows for the two most technically significant workflows — engine sound analysis and dashboard warning light scanning — demonstrating how the system's modular architecture supports both on-device offline operation and cloud-assisted enhancement. Finally, the Deployment Architecture described the five-tier physical infrastructure and articulated the design rationale behind key decisions such as on-device inference, cloud service delegation, and the independent ML Model Update Server.

The architectural principles that underpin CheckAR — modularity, separation of concerns, scalability, security, and offline resilience — ensure that the system can evolve in response to future requirements. The ML model update mechanism allows diagnostic accuracy to improve continuously as new training data becomes available. The extensible class and component architecture accommodates the addition of new input modalities (such as OBD-II connectivity), new diagnostic domains (such as tyre condition analysis), or expanded vehicle manufacturer support without requiring fundamental redesign. The microservice-oriented backend supports independent scaling of individual service components as the user base grows.

In conclusion, the architectural documentation presented in this report provides a rigorous, well-reasoned, and implementable specification for the CheckAR system. It serves as the authoritative technical reference for the development team during implementation, the quality assurance team during testing, and project stakeholders throughout the deployment and maintenance lifecycle of the application.

9. References

The following references informed the design, architectural decisions, and documentation approaches applied in this report:

[1] Pressman, R. S., & Maxim, B. R. (2015). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill Education.

[2] Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education Limited.

[3] Fowler, M. (2003). UML Distilled: A Brief Guide to the Standard Object Modeling Language (3rd ed.). Addison-Wesley Professional.

[4] Google LLC. (2024). Firebase Documentation — Authentication, Firestore, Cloud Messaging. Retrieved from https://firebase.google.com/docs

[5] Google LLC. (2024). Google Maps Platform Documentation. Retrieved from https://developers.google.com/maps

[6] Google LLC. (2024). YouTube Data API v3 Documentation. Retrieved from https://developers.google.com/youtube/v3

[7] TensorFlow Authors. (2024). TensorFlow Lite — Machine Learning for Mobile and Edge Devices. Retrieved from https://www.tensorflow.org/lite

[8] Flutter Development Team. (2024). Flutter Documentation — Building Beautiful Native Applications. Retrieved from https://flutter.dev/docs

[9] OpenJS Foundation. (2024). Node.js Documentation. Retrieved from https://nodejs.org/en/docs

[10] Object Management Group. (2017). Unified Modeling Language Specification, Version 2.5.1. Retrieved from https://www.omg.org/spec/UML/2.5.1

[11] Larman, C. (2004). Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design (3rd ed.). Prentice Hall.

[12] DeMarco, T. (1979). Structured Analysis and System Specification. Prentice Hall.